Software Engineering coursework specification

1) Introduction

Watson Games is a company that produces a range of traditional board games. You have been appointed as a software contractor to develop a computer game version of their most popular title Clue!, a murder mystery game similar to the classic Cluedo. Watson Games have never commissioned such a system before and so do not have a formal requirements document to specify how the software should operate. They do, however, have over 50 years of experience developing and marketing board games and have put together a set of user requirements to describe, in their own terms, what they want the system to do. This document contains all their user requirements. It is possible these requirements might evolve as the project progresses, but the core themes of the requirements will not change. You should, however, design with flexibility in mind.

Working as a team, you will need to:

- Develop an application following a form of the Agile development process. This will involve several phases of design, development, and testing. This may involve, as you see fit, determining sets of functional requirements, non-functional requirements and domain requirements to determine more precisely what the software system should do. This may involve going back to the client to clarify exactly what they want, or presenting the client with a range of options to help them develop their own ideas.
- Develop an appropriate software design, expressed using an appropriate combination of high and low level design techniques including, although not limited to, class diagrams, entity relationship diagrams, use case diagrams, sequence diagrams, state transition diagrams, activity diagrams, other UML or UML style/derived diagrams.
- Write the software, using whatever tools, development environments and languages you deem appropriate. Any reasonable set of choices will be accepted, including, although not limited to, Java, JavaScript, C, C++, Scala, PHP, Python and Unity. You may opt for a system using a database backend for persistent data management if you wish. Whatever you think is needed and that you are prepared to justify as part of the design process. The software will include a Graphical User Interface (GUI) component to help you monitor progress of the software simulation. You should bear in mind that the choice of tools and languages is something you should justify, and it must be a choice that is inclusive for all members of your team that will be working on the development of your codebase.
- Develop a test plan for the software, both at a unit testing level and at a system level to demonstrate that your software meets all the user requirements set for it.
- Write a report to document the work that you have done.
- Give a demonstration of your completed software system.
- Demonstrate that you can work as a coherent team, and show that you understand the significance of software engineering principles in the development of a non-trivial piece of software.

Assessment criteria for this team coursework project can be found on Canvas. No two teams will produce systems that work exactly the same, so there is no gold standard for the operation of the software that you will be assessed against. The assessment criteria will focus on the quality of your software engineering skills, and your ability to function as a team, to plan and execute work in an appropriate manner and deliver a working and hopefully reliable software system.

You will work in a team of 5 people. You will have a short initial window to form your own teams. If you have not formed a team by the end of that window period, your will be assigned to a team. In that event, there will be no negotiation on who is assigned to your team. Get to know the other students in your team. The assessors reserve the right to amend team membership (but this would be in exceptional cases only), and to merge teams if their membership becomes sub-critical, for example if students drop out from the course for personal reasons.

As this specification represents a set of user requirements, it has been expressed in user-centric terms. It may be incomplete or contradictory in places. Technical terms may be used inconsistently. If there is something you do not understand, you can either ask the customer's representative (course tutors/teaching assistants) or make reasonable assumptions about what is meant. Note that in the former case, the customer may or may not have a view on the query, and in the latter case you should be prepared to justify the choices and decisions you have made.

2) The client: Watson Games

Formed in 1963, Watson Games Ltd are the UK's leading producer of quality board games. In 1986, they expanded upon the acquisition of Wadleys after they ran into financial difficulties arising from the widely reported corporate espionage scandal "Mousetrapgate". Watson Games design, produce and market high end board games designed to appeal to a wide range of ages.

The shareholders for Watson Games feel that the company needs to expand its operations to embrace the digital era and so have decided to develop a computer version of their classic Clue! game. Clue! was originally designed for up to 6 players and has a reputation as a social experience. However, Watson also realise that it is often not possible in an ever-complex world to find 6 players to play the classic game. The head of game development, Quentin Raffles, has hit upon the idea that the computer version (the "simulation") could include an option for the computer program to take the role of one or more of the players, offering an opportunity for a richer gaming experience for a smaller number of players.

3) The rules of Clue!

Quentin Raffles has provided the rules as they currently exist for classic Clue!.

1) The game is for 2-6 players. The game is played by moving around a playing board that represents the ground floor of Archers Avenue. The object of the game is to solve, by means of elimination and deduction, the problem of the murder of Dr Phlox, the owner of the house, whose body has been found at the bottom of the stairs. The winner is the first person to identify in one accusation the murderer, the weapon used and the room in which the crime was committed. That information is stored on cards placed in the "murder envelope" and remains a mystery until a player makes the correct accusation.
2) The 6 persons in the house are represented by playing pieces: Col Mustard, Prof Plum, Rev Green, Mrs Peacock, Miss Scarlett and Mrs White. The weapons are represented by small tokens. Any one of the weapons might have been the one used. The weapons are as follows: dagger, candlestick, revolver, rope, lead piping and spanner.
3) There are 6 cards representing the persons in the house. There are 6 cards representing the weapons. There are 9 cards representing the rooms shown on the playing board. In the physical version there are "detective notes cards" to enable players to record their investigations.
4) The game is setup as follows:

- The playing pieces are placed on the starting squares marked for them on the board. If less than 6 people play (including autonomous players), those playing pieces and their cards not represented <u>should not</u> be removed from the game. Spare pieces are placed in any room or rooms before the game starts whilst the cards remain in the pack and are subsequently dealt to players.
- Each of the weapons in placed in a different room.
- The envelope marked “murder cards” is placed on the spot marked X on the board,
- The room cards are shuffled. The top card is placed unseen into the murder envelope.
- The weapon cards are shuffled. The top card is placed unseen into the murder envelope.
- The person cards are shuffled. The top card is placed unseen into the murder envelope.
- All remaining cards are placed face downwards on the table. Each player shakes the dice and the highest number is determined to be the dealer.
- The dealer shuffles the 18 remaining cards. The cards are dealt to players one at time clockwise around the table, including the dealer.
- Some players may receive more cards than others. This is not an issue.

5) Each player adopts the playing piece nearest to them and is that character for the rest of the game.
6) By tradition, Miss Scarlett goes first. The player shakes the dice and moves that number of squares along the passage to any room they like. Players take turns in a clockwise direction.
7) When a player reaches a room, they can call out a “suggestion” by calling into that room any other person and any weapon. For instance, if Miss Scarlett was in the lounge, she may call Rev Green and the spanner into that room. She would then suggest that the murder was committed “in the lounge by Rev Green using the spanner”. Spare playing pieces fall under equal suspicion and can called into rooms by players making suggestion.
8) When the suggestion is made, starting from the current player’s left, if they have one of the cards mentioned in the suggestion, then they must show one only of those cards to the current player (unseen by other players). If the player on the left is unable to show a card, then we move to the next player to the left until a player is able to show a card to the current player. Once that happens, or all players have been tried leading back to the current player, then the turn of the current player is ended.
9) Eventually, and by using the detective cards, players should be able to eliminate all but the true murderer, room and a weapon.
10) When a player thinks they know the 3 murder cards, then they can make an “accusation” directly after making their suggestion. When an accusation in made, the player examines, unseen by other players, the cards in the murder envelope. If the accusation is proven correct, the player wins the game.
11) A player can only make one accusation. If the accusation is incorrect, the murder cards are returned to the murder envelope. The player has no further turns in the game, but remains as a player only to contradict suggestions made other players, with the cards they hold in their hand.
12) Skilled player may make bluff suggestion by including a person, weapon of room for which they hold the card. This can be used to determine the whereabouts of one card and to mislead other players.

13) Players may move their pieces anywhere on the board on the squares according to the number thrown on the dice. Players must move in straight lines only i.e. forwards and cross wise, but never diagonally.
14) Players may enter rooms by their doors only, but cannot enter and leave a room in the same move Entering a room ends the move count. If a player throws a 6 and only needs 4 to enter a room, they may enter the room and ignore the 2 moves left over.
15) No two pieces may occupy the same square. A room however may be occupied by any number of pieces and weapons. A player may choose to remain stationary during any number of turns in play.
16) Secret passages enable players to move from certain rooms to those indicated on the board. This can be done on a player's turn without throwing the dice, and constitutes the move.
17) A player may make a suggestion only when their playing piece is in the room mentioned in the suggestion.
18) Playing pieces and weapons transferred to any room as a result of a suggestion are not returned to their original positions afterwards. Players must continue playing their pieces from their new locations.
19) Watson games are working on an updated pack for their classic board with special card features for certain squares. But the details of this update have not yet been decided.

The game board

The classic game board is as shown below. It can easily be found online also. Although this is the classic, Watson Games think that the electronic version offers the possibility of board customisation by the user.

4) Other user requirements

As Watson Games have no experience with software development, all they can do is to provide some general statements in respect of what they hope to achieve:

- The electronic version should be for desktop machines, and ideally should be playable on both Mac and PCs. If this is difficult, then PC development should be preferred.
- There are no plans for a mobile version at this stage.
- The game should be fun to play and have a colourful and intuitive interface that reflects the spirit and character of the original board game.
- With the provision of the autonomous player agent feature, it should be possible for a full experience game to be enjoyed by just a single human player.

5) Core elements of the software system

You will need to design and implement a number of key elements to make this simulation work. As well as all the necessary classes, code, interface GUI and other infrastructure elements, you will need to create:

- **A game player agent:** An agent that can take the role of 1 or more of the players. This would allow for a limited number of human players to enjoy a richer gaming experience. However,

it also provides the possibility for fully autonomous play when all the players are provided by the program. This will enable you to investigate various strategies as to how to play the game to the best advantage. Such simulations could be performed at high speed. Such simulations also offer a means of testing the performance and correct operation of your game.

You will not be assessed against how well the autonomous game agent performs in purely detective terms. In the first instance the game player agent could just be based on random decision making.

You will also need to ensure that your simulation has:

- **A means of uploading initial data:** to get the simulation started you will need a means of initialising it with data on the board layout, the players and other data need to make the game function. As this data will be loaded on start-up from external files, this means that the game is easily customised, and Watson Games see this as a valuable selling point of the new electronic version.
- **A means of monitoring the performance of the simulation:** an appropriate GUI to enable players to see and access all data that they would in the classic board game.
- **A means of being tested:** to ensure that the market is operating properly in accordance with the rules of the game, and to demonstrate that the software is working correctly

It falls to your team to determine the form, structure and means of implementation of any user interface that your system will need to meet the requirements. It should be noted, however, that a flashy GUI does not compensate for poor gameplay.

6) The game player agent

The game player agent should be able to play the game to the same extent that a human player would. The game player agent would roll moves, make suggestions and an accusation. The key point here is that the game player agent needs to be able to play the game, but it does not necessarily need to be any good at it, at least in the first instance.

A game player could incorporate some simple rule for making in game decisions, or could feature relatively sophisticated Artificial Intelligence. But a game player could just operate on purely random decision making. That would still allow the game to be played. It is suggested that your make the game player agent perform random decision making initially, and then if time permits, look to increase the sophistication of the agent. The marking scheme places emphasis on the delivery of a working agent, and only a small amount of its relative game playing sophistication.

Players may not retire from the game.

7) Integrity of the game.

In the board game version, it is the duty of players to disclose cards to the current player when required by the rules. Cheating is just not acceptable.

The dice used in the game must be fair with each dice have an equal probability of landing on one of its six sides.

Some information needs to remain private between players – this is essential to the correct working of the game This will have an impact on the design of any GUI that you implement. Watson Games have no particular ideas as to how the GUI should look – they are looking for ideas from you. You can regard this project as producing a prototype for Watson Games to consider. As such, any reasonable

approach to the GUI design that meets the needs of the game and its rules is acceptable. This will depend, to a large part, on how you decide to implement the game.

8) Deliverables

You are responsible for the delivery of this project as a group. Submission will be via the Esubmissions system on Canvas. You will need to produce:

- **A project plan:** showing the tasks that you identified for your team, how much effort you planned for each task, when it was scheduled to start and complete, and which team member(s) were responsible for it. This will likely take the form of a PERT style analysis or a Gantt chart. Bear in mind that the execution of the plan may well up end quite different from the initial plan. This is quite normal as sometimes things turn out differently to what you expect. If the variations are small, you don't need to keep the plan up to date. If the execution is widely different from the plan, you should consider updating the plan and presenting both the initial and revised plans in your submission, together with commentary to explain why the plan fell apart.
- **Process documents**: This project will be developed over a series of incremental developments ("sprints" in the Agile terminology). Your process documents should record what you did for each incremental phase. There is a template sprint reporting document on Canvas. You need to complete a version of this document for each sprint. Details of what you be in each sprint process document is shown below.
- **The code base:** including all non-code files or other data necessary to allow your submission to be compiled and executed by the assessors. The code should be properly documented (e.g. using Javadocs if you are working in Java), and be commented to an appropriate standard. As well as the completed codebase, you should retain a snapshot of your codebase at the end of each incremental development phase as set out in your process document. Remember that a sprint cycle should result in a working program with some value.
- **A group report**: documenting how the project went for you, with a critical analysis of the performance of your team, and describing what worked well for your team and what did not. The report must also contain records of any group meetings that you had, together with a summary of actions agreed and a weekly log recording your progress.
- **A peer assessment:** A document where the team agrees the distribution of a pool of marks to reflect the relative contributions of each team member to the group project as a whole. Your individual mark for this Software Engineering module will be a composite of your team score and an individual score determined by your peer assessment. If any individual does not make a contribution to the group effort, the assessors reserve the right to award an individual score of 0 for that team member for the module as a whole.
- **A video:** Providing evidence of the final working state of your program. The purpose of this video is to assist the markers in evaluating the working state of your final delivered program, and to enable you to showcase any particular features that you are particularly pleased with.

The process documents should contain documentation for each of your Agile sprints. The process document is not intended to be a heavyweight account of the sprint, but it must provide evidence of the work that you have done. There is a template document on Canvas. The sprit cycle process document for each individual sprint should record:

- **The sprint objectives:** the target objectives, user requirements or user stories you set for yourself, any functional, non-functional or domain requirements that you identified that

needed to be addressed, any other reasonable assumptions that you made as part of any requirements analysis process.

- **Design:** documenting both high level and low-level designs of your software systems. Such a document should include, although is not limited to, class diagrams, sequence diagrams, activity diagrams, state transition diagrams, use case analyses and other UML style diagrams. The design does not need to be exhaustive. You only need to do as much design as your team deems necessary to deliver each of the incremental development phases.
- **Testing**: showing how you tested the software at both unit test and system test levels. It is acceptable for bugs and other issues to remain in the software as long as they are clearly identified, documented and that you have provided appropriate commentary to explain why they exist and what you would do about them.
- **Post sprint review:** A review of how successful the sprint was in achieving the objectives you set out for it.

Your team will also be required to attend a progress review meeting with one of the assessors to ensure that you are keeping on track. You can request additional meetings if you need them and the assessors will endeavour to provide support as needed. But your first port of call for any problems should be the scheduled seminar sessions.

When you upload your deliverables, please use Word or PDF formats for text documents. Do not use Mac Pages files as these are not widely supported. For code deliverables, please remember to include any project (e.g. Eclipse, BlueJ), initialisation or other external files your code needs to be run. If assessors cannot run your code, they will likely ask for demonstrations of the operation of key parts of your code. Do not upload files that are in obscure formats. Assemble all of your documents together and create a single ZIP file with everything inside. Upload the zip file onto the Esubmissions system. Check that your ZIP file unpacks correctly before uploading it.

You may also be required to give a short demonstration of your project at the end of the semester. This will be an informal demonstration to your lab seminar leaders or the module convenor.

9) Peer assessment

Each team has 20 points to distribute per member of the group. Each member gets a peer mark, which is a non-negative integer (including 0) such that the sum of all members' peer marks is 20 x the number in the group. The idea is that the peer mark reflects the contributions of each member to the project. It is perfectly acceptable to give each member the same mark.

Here is an example. Assume the group has members: Alice, Bob, Charles, Diana, Eric, Fiona, George and Herbert.

- Alice: 25
- Bob: 0
- Charles: 30
- Diana: 20
- Eric: 15
- Fiona: 15
- George: 35
- Herbert: 5

In this case, George contributed most to the group, followed by Charles and Alice, and so on. Bob is a "ghost" - who never contributed to the group work at all. I take that as a signal from the rest of the group to that effect, and he will not be awarded the grade for the project the others receive. I think that's fair. Note that a ghost's marks count as some measure of compensation for the others for the increased workload they are under.

When submissions are graded, each member of the team receive a base mark (except for ghosts) that reflects the quality of the design process, how well teamwork has gone on, and so on (actual code, while it inevitably has an influence, is the least important element from the point of view of this module). The team score will then be combined with the peer mark. A standard peer mark of 20 will count for around 10 percentage points, or about one grade.

Only one submission per team of your agreed marks is required, but it will need to be clear that it is agreed between you (excluding ghosts). If there is really no agreement, I will make the final decision. To do so, the team will be called to a follow up meeting to explain the reason(s) for their disagreement.

10) Final observations

There is no single right way of designing and delivering this Software Engineering assignment. You will be assessed according to your ability to come up with a coherent set of requirements, project plan, software design, implementation, testing and production of a final report. A set of marking criteria will be published on Canvas.

If you do not understand something, use the Software Engineering seminars to ask questions and raise discussions on the assignment. You can also discuss ideas with other teams, but do not copy the work of other teams. If you require clarification on any points arising from this document, any such clarification will be copies to all students and teams via your Sussex email address. You should also keep an eye on your Sussex emails and announcements on the Canvas module page in case the Watson Games project manager decides to issue an updates, amendments or clarifications on their user requirements. The customer is always right, even when they are changing their mind. As a consequence, you should try to anticipate any requirements shift that might arise and try to design your software so that changes can be easily accommodated. This is not always an easy thing to achieve in practice.

You will need to ensure that you team meets often, documents decisions that it makes and that the workload is spread fairly between team members. The most common cause of problem with team based assignments such as this is a breakdown in team dynamics. A part of the assessment process will require teams to submit a peer assessment to evaluate the relative contributions of each team member to the overall delivery. If a team member does not contribute to the team effort as a whole, I reserve the right to award that team member a score of 0. Do not leave problems unresolved. If you are having team issues, you should try to resolve them yourselves first, and take the issue up with one of the teaching assistants in the seminars, or through a private meeting between one of the teaching assistants or me as you see appropriate.

So, work together, and have some fun working on this group coursework.

And keep regular backups of your work ……. “I lost the USB stick” just doesn’t cut any ice.

Hsi-Ming Ho
hh435@sussex.ac.uk
January 2026

END.